

Proyecto de curso: Programación Vectorial en Ensamblador x86-64 (NASM/Linux)

Josué Rodríguez Palacios, Sary Vargas Zamora
 josue2002rodriguez@estudiantec.cr, sarvargas@estudiantec.cr
 Escuela de Ingeniería Electrónica
 Instituto Tecnológico de Costa Rica
 Arquitectura de Computadoras

Resumen—Este proyecto presenta una comparación entre una implementación escalar y una implementación vectorizada en ensamblador x86-64 con NASM/Linux para el cálculo de estadísticas y normalización de arreglos de datos. La versión vectorial utiliza AVX2 y registros YMM de 256 bits para procesar ocho valores `float32` simultáneamente. La correctitud se valida frente a una referencia en Python y mediante comparaciones directas entre las implementaciones, incluyendo casos borde y entradas con valores constantes o de gran magnitud. En las pruebas de rendimiento, la versión vectorial reduce el tiempo de ejecución del kernel en todos los tamaños evaluados y alcanza un speedup máximo reportado de $13.11\times$ para $N = 10^5$. Para $N = 5 \times 10^7$, también se observan reducciones en ciclos e instrucciones totales según los contadores de hardware.

Palabras clave—AVX2, SIMD, x86-64, NASM, ensamblador, vectorización, registros YMM, rendimiento, GDB, perfilado de hardware.

I. INTRODUCCIÓN

LOS procesadores x86-64 modernos incorporan extensiones SIMD que permiten ejecutar una misma operación sobre múltiples datos en paralelo. En este proyecto se analiza esta capacidad comparando entre una implementación escalar y una implementación vectorizada en lenguaje ensamblador NASM para GNU/Linux, utilizando instrucciones AVX2 y registros YMM. El propósito es analizar tanto el correcto funcionamiento de las rutinas como el efecto de la vectorización sobre el tiempo de ejecución y los contadores de rendimiento del procesador.

La implementación escalar procesa los elementos de manera secuencial, mientras que la versión vectorial permite procesar varios datos al mismo tiempo para trabajar con bloques de ocho valores de punto flotante de precisión simple. Para comprobar que ambas versiones den resultados equivalentes se comparan los resultados con una referencia en Python y se evalúan casos de prueba que incluyen tamaños pequeños, valores constantes y valores grandes. Posteriormente, se analizan los tiempos de ejecución y de los datos obtenidos con el perfilador de hardware.

El documento presenta primero el entorno de pruebas, seguido de las implementaciones escalar y vectorial, la validación de correctitud, los resultados de rendimiento y la evidencia obtenida mediante GDB. Finalmente, se resumen las conclusiones y se incluye el registro completo de las pruebas en el apéndice.

II. DESCRIPCIÓN DEL ENTORNO DE PRUEBAS

Para la compilación de los códigos fuente, el análisis de rendimiento y la depuración a nivel de registros, se utilizó un entorno de trabajo sobre un sistema operativo GNU/Linux de 64 bits. El hardware de pruebas cuenta con un procesador Intel Core de duodécima generación, el cual cuenta con soporte para el conjunto de instrucciones de vectorización avanzada AVX2 (Advanced Vector Extensions 2), permitiendo el procesamiento de varios valores `float32` al mismo tiempo (32 bits) en registros de 256 bits ('YMM').

Las características del equipo utilizado, junto con las versiones exactas de las herramientas de compilación, control de flujo, depuración y perfilado de hardware, se mencionan en la Tabla I.

Cuadro I
HERRAMIENTAS Y ENTORNOS UTILIZADOS

Componente / Herramienta	Especificación / Versión
Procesador (CPU)	Intel Core i5-12450H
Arquitectura	x86_64 (Soporte AVX2, FMA)
Sistema Operativo	Ubuntu Linux (x86_64)
Ensamblador	NASM v2.16.01
Compilador C	GCC v13.3.0 (-O2, gnu11)
Depurador	GNU GDB v15.1
Perfilador	Linux perf v7.0.14

III. DIAGRAMAS

Como complemento a la descripción de las implementaciones desarrolladas, se elaboró un documento que reúne los cuatro diagramas solicitados para las versiones escalar y vectorial. Estos incluyen el diagrama de arquitectura de software, el flujo de control de los kernels, la asignación de registros y el mapa de memoria y alineación.

El documento completo de diagramas se encuentra disponible en el repositorio del proyecto en el siguiente enlace:

Diagramas de arquitectura escalar y vectorial

IV. IMPLEMENTACIÓN ESCALAR

La implementación escalar que se realizó procesa los datos de manera secuencial, elemento por elemento, utilizando registros de propósito general y registros de punto flotante. Esta arquitectura utiliza la convención de llamadas estándar para comunicarse correctamente con el código en C en entornos

Unix/Linux [1]. Se utilizó el código base brindado por el profesor y se agregaron las funciones de compute stats y normalize_array como se muestra a continuación:

A partir del código base brindado por el profesor, se completaron las funciones compute_stats y normalize_array. La primera realiza el cálculo de la media, varianza, mínimo y máximo del arreglo, mientras que la segunda genera el arreglo normalizado mediante la expresión $y[i] = (x[i] - \mu)/\sigma$.

Fragmento de compute_stats

```
.mean:
    call    sum_array
    cvtsi2ss xmm1, r12d
    divss   xmm0, xmm1
    movss   [r13], xmm0

    movss   xmm3, [rbx]
    movss   xmm4, [rbx]
    movss   xmm2, xmm0
    xorps   xmm0, xmm0
    xor     eax, eax

.stats_loop:
    cmp     eax, r12d
    jge     .operation_done

    movss   xmm1, [rbx + rax*4]
    minss   xmm3, xmm1
    maxss   xmm4, xmm1
    subss   xmm1, xmm2
    mulss   xmm1, xmm1
    addss   xmm0, xmm1

    inc     eax
    jmp     .stats_loop

.operation_done:
    cvtsi2ss xmm1, r12d
    divss   xmm0, xmm1
    movss   [r14], xmm0
    movss   [r13], xmm2
    movss   [r15], xmm3
    pop     r9
    movss   [r9], xmm4
```

Fragmento de normalize_array

```
normalize_array:
    test    edx, edx
    jle     .done

    movss   xmm8, xmm0
    movss   xmm9, xmm1

    xor     eax, eax
    xorps   xmm0, xmm0
    xorps   xmm1, xmm1

.zero_stddev:
    comiss  xmm0, xmm9
    je      .zero_case_stddev

.loop_norm:
    cmp     eax, edx
    jge     .done

    movss   xmm1, [rdi + rax*4]
    subss   xmm1, xmm8
    movss   xmm0, xmm1
    divss   xmm0, xmm9
    movss   [rsi + rax*4], xmm0
    inc     eax
    jmp     .loop_norm

.zero_case_stddev:
    cmp     eax, edx
    jge     .done

    movss   xmm0, [rdi + rax*4]
    movss   [rsi + rax*4], xmm0
```

```
inc     eax
jmp     .zero_case_stddev

.done:
    ret
```

Del código anterior, se observa lo siguiente:

- **Preservación de Registros (callee-saved):** De acuerdo con la convención de llamadas System V AMD64 ABI, los registros rbx, rbp y r12-r15 deben ser preservados por la subrutina si van a ser modificados [1]. En compute_stats, estos registros se guardan al inicio en la pila porque se realiza una llamada auxiliar (call sum_array), la cual altera los registros volátiles (callee-saved), para conservar los punteros de memoria y el tamaño del arreglo originales [1].
- **Manejo de casos borde ($N \leq 0$):** Se valida el tamaño mediante test y jle para evitar accesos inválidos a memoria y divisiones por cero a nivel aritmético, asignando cero a los punteros de salida.
- **Cálculo de varianza, mínimo y máximo en el mismo recorrido:** El bucle .stats_loop optimiza el rendimiento reutilizando los datos cargados en registros para actualizar simultáneamente los extremos (minss/maxss) y acumular la varianza poblacional, minimizando el impacto de los ciclos de espera de memoria (memory latency stalls) [3].
- **Manejo de división por cero:** En normalize_array, el uso de comiss permite evaluar condiciones de borde aritméticas cuando la desviación estándar es nula ($\sigma = 0.0$), saltando a una rutina que copia los datos directamente que evita una división por cero [2].

V. IMPLEMENTACIÓN VECTORIAL

La versión vectorizada aprovecha AVX2 para procesar varios datos al mismo tiempo haciendo uso de extensiones AVX2 (Advanced Vector Extensions 2), permitiendo procesar bloques de 8 datos en punto flotante (float32) de forma simultánea en registros de 256 bits (YMM0–YMM15) [2].

A continuación, se muestra la implementación de las funciones compute_stats y normalize_array:

Fragmento de compute_stats

```
; Media replicada en los 8 carriles
vbroadcastss ymm6, xmm0

; Acumulador vectorial de varianza
vxorps ymm0, ymm0, ymm0

mov     ecx, r12d
and     ecx, ~7
xor     eax, eax

.loop_vectorization:
    cmp     eax, ecx
    jge     .stats_reduce

    vmovaps ymm1, [rbx + rax*4]

    vminps  ymm4, ymm4, ymm1
    vmaxps  ymm5, ymm5, ymm1

    vsubps  ymm3, ymm1, ymm6
    vmulps  ymm3, ymm3, ymm3
    vaddps  ymm0, ymm0, ymm3

    add     eax, 8
```

```

    jmp     .loop_vectorization

.stats_reduce:
    vextractf128 xmm2, ymm0, 1
    vaddps  xmm0, xmm0, xmm2
    vhaddps xmm0, xmm0, xmm0
    vhaddps xmm0, xmm0, xmm0

.scalar_operation:
    cmp     eax, r12d
    jge     .scalar_operation_done

    vmovss  xmm1, [rbx + rax*4]
    vminss  xmm3, xmm3, xmm1
    vmaxss  xmm4, xmm4, xmm1

    vmovss  xmm5, [r13]
    vsubss  xmm1, xmm1, xmm5
    vmulss  xmm1, xmm1, xmm1
    vaddss  xmm0, xmm0, xmm1

    inc     eax
    jmp     .scalar_operation

```

Fragmento de normalize_array

```

    vbroadcastss ymm1, xmm8
    vbroadcastss ymm2, xmm9

    mov     ecx, edx
    and     ecx, ~7
    xor     eax, eax

.loop_norm:
    cmp     eax, ecx
    jge     .norm_scalar

    vmovaps ymm4, [rdi + rax*4]
    vsubps  ymm4, ymm4, ymm1
    vdivps  ymm4, ymm4, ymm2
    vmovaps [rsi + rax*4], ymm4

    add     eax, 8
    jmp     .loop_norm

.norm_scalar:
    cmp     eax, edx
    jge     .norm_done

    vmovss  xmm0, [rdi + rax*4]
    vsubss  xmm0, xmm0, xmm8
    vdivss  xmm0, xmm0, xmm9
    vmovss  [rsi + rax*4], xmm0

    inc     eax
    jmp     .norm_scalar

```

Del código anterior, se observa:

- Replicación del valor con `vbroadcastss`: La instrucción `vbroadcastss` duplica un valor escalar en los 8 carriles de un registro vectorial YMM, permitiendo aplicar la misma operación sobre los 8 valores (`vsubps`, `vmulps`, `vdivps`) sin requerir bucles anidados [2].
- Reducción Horizontal de Registros Vectoriales: Para combinar los 8 resultados parciales en un solo valor, se combinan extracciones de mitades altas (`vextractf128`), sumas horizontales (`vhaddps`) y máscaras de mezcla de bits (`vpshufd`), optimizando la latencia de cómputo en la unidad aritmética [2, 3].
- Manejo del Remanente: El límite del procesamiento vectorial se obtiene con la operación `and ecx, ~7`, que determina el mayor múltiplo de 8 menor o igual que N . Cuando la cantidad de elementos no es múltiplo de 8, los valores restantes se procesan individualmente mediante un bucle escalar de cierre.
- Prevención de Penalización AVX/SSE (`vzeroupper`):

Conforme a las recomendaciones de Intel, la instrucción `vzeroupper` se ejecuta antes de cualquier retorno (`ret`) para limpiar la parte superior de los registros YMM, evitando penalizaciones de rendimiento asociadas a los cambios de contexto entre instrucciones AVX y instrucciones SSE [2].

V-A. Casos de prueba

Al realizar las pruebas con distintos casos de N y al realizar la comparación con el archivo de python que calcula los mismos parámetros, se obtuvo que todas las pruebas pasan, dado que están dentro del rango establecido, como se observa en la tabla II.

Cuadro II
RESUMEN DE VALIDACIÓN DE CORRECTITUD PARA LOS CASOS DE PRUEBA.

Caso	N	Escalar vs. Python	Vectorial vs. Python	Escalar vs. Vectorial
input_0.dat	0	PASA	PASA	PASA
input_1.dat	1	PASA	PASA	PASA
input_7.dat	7	PASA	PASA	PASA
input_8.dat	8	PASA	PASA	PASA
input_15.dat	15	PASA	PASA	PASA
input_16.dat	16	PASA	PASA	PASA
input_1000.dat	1000	PASA	PASA	PASA
input_const.dat	100	PASA	PASA	PASA
input_edge.dat	100	PASA	PASA	PASA

Además, en el anexo se adjuntan todos los resultados obtenidos para cada caso, con lo que se comprueba que se maneja correctamente los casos donde no hay elementos o hay un solo elemento, evitando divisiones por cero o fallas de segmentación. También que las diferencias relativas con respecto a la referencia en Python se mantienen en valores muy bajos (10^{-7} a 10^{-10}), lo cual confirma que el uso de aritmética de punto flotante de precisión simple de 32 bits en los registros XMM/YMM preserva la precisión esperada en arquitecturas x86-64.

Por otra parte, en casos como `input_const.dat` (valores repetidos) e `input_edge.dat` (valores grandes del orden de $\pm 1\,000\,000$), las rutinas calcularon una varianza y desviación estándar coherentes, demostrando que los acumuladores de suma y cuadrados resisten la saturación numérica moderada en precisión simple.

V-B. Resultados rendimiento

Se comparó el rendimiento de la versión escalar y la vectorial, en la tabla III y gráfica 1, se muestra lo obtenido durante la ejecución de ambas versiones. Se observa que en la versión vectorial se reduce el tiempo de ejecución del kernel en todos los tamaños evaluados (alcanzando un speedup de hasta $\sim 13\times$ en $N = 10^5$). Esto ocurre porque las instrucciones AVX2 procesan múltiples elementos en paralelo utilizando registros de 256 bits, disminuyendo el número total de iteraciones y de instrucciones ejecutadas a nivel de procesador ($\sim 3.65 \times 10^9$ instrucciones escalares frente a $\sim 1.66 \times 10^9$ vectoriales).

Para probar un valor grande de N , se obtuvo el rendimiento con N valor de 5×10^7 . En la tabla IV, se observa el consumo de recursos.

Cuadro III
COMPARATIVA DE TIEMPO DE EJECUCIÓN DEL KERNEL Y SPEEDUP POR
TAMAÑO DE ENTRADA (N).

Tamaño (N)	Escalar (ms)	Vectorial AVX2 (ms)	Speedup
10^3	0.0067	0.0012	5.58×
10^5	0.5128	0.0391	13.11×
10^6	5.0691	0.5574	9.09×
5×10^7	259.9816	48.8024	5.33×

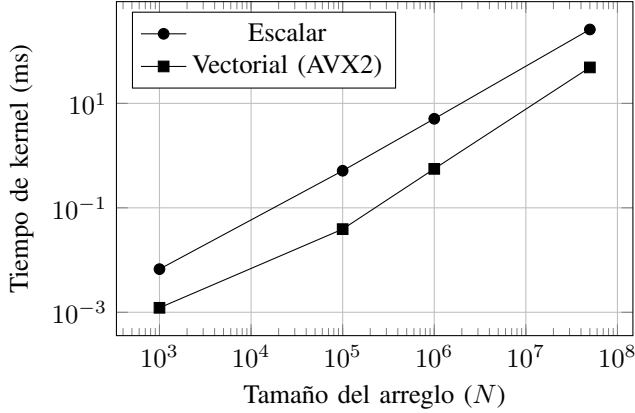


Figura 1. Comparativa de tiempos de ejecución entre la implementación escalar y vectorial en función de N .

También se obtuvieron datos con el perfilador de hardware, sumando los contadores de los núcleos de rendimiento (cpu_core) y eficiencia (cpu_atom), los resultados se muestran en la Tabla V.

De las tablas anteriores, se observa que, la versión vectorial requiere casi la mitad de ciclos de reloj totales para procesar los 50 millones de elementos, lo que evidencia un uso más eficiente del pipeline del procesador. También, la implementación vectorial muestra una reducción en los fallos de caché gracias a la alta localidad espacial y al aprovechamiento óptimo de los bloques de datos cargados en los registros SIMD de forma contigua. Además, es importante mencionar que el uso de RAM se mantiene igual, ya que se maneja la misma estructura de datos de entrada y salida.

Cuadro IV
CONSUMO DE RECURSOS DEL SISTEMA PARA $N = 5 \times 10^7$ (381.47 MB).

Métrica de Recurso	Versión Escalar	Versión Vectorial (AVX2)
Tiempo de Usuario (User time)	0.27 s	0.06 s
Tiempo de Sistema (Sys time)	0.19 s	0.20 s
Uso de CPU (% CPU)	22 %	11 %
Memoria Residente Máxima (RSS)	392,404 KB	392,412 KB
Intercambio de Páginas (Swaps)	0	0

Cuadro V
CONTADORES DE RENDIMIENTO DE HARDWARE OBTENIDOS MEDIANTE
PERF STAT PARA $N = 5 \times 10^7$.

Contador de Hardware	Versión Escalar	Versión Vectorial (AVX2)
Ciclos Totales (Core + Atom)	$\approx 3.46 \times 10^9$	$\approx 1.82 \times 10^9$
Instrucciones Totales	$\approx 3.65 \times 10^9$	$\approx 1.66 \times 10^9$
Referencias a Caché (Cache-refs)	76.12×10^6	57.16×10^6
Fallos de Caché (Cache-misses)	57.80×10^6	48.32×10^6
Tiempo Transcurrido (Wall-clock)	2.28 s	2.40 s

V-C. Evidencia de la sesión de GDB

Con el propósito de comprobar la correcta manipulación de los registros de 256 bits y el alineamiento de memoria en la arquitectura x86-64, se realizó una sesión de depuración interactiva utilizando GNU Debugger (GDB). La Figura/Listado ?? detalla la traza de ejecución para un tamaño de prueba reducido ($N = 16$), mostrando dos puntos críticos de control:

1. **Inspección de Registros Vectoriales** : Al detener la ejecución en la rutina `compute_stats()`, se inspectó el registro `ymm0` justo después de cargar un bloque de datos. El despliegue confirma que el empaquetado de punto flotante de 32 bits opera correctamente, visualizando los cuatro primeros elementos duplicados/cargados en paralelo y las posiciones superiores en cero.
2. **Inspección de Memoria del Arreglo** : Mediante el puntero de argumento alojado en el registro `rdi`, se examinó la memoria contigua en la rutina `normalize_array()`, verificando que los valores de entrada flotantes se encuentran correctamente posicionados y listos para ser procesados por las instrucciones SIMD.

Sesión de depuración y captura de registros/memoria en GDB para $N = 16$

```
=====
INICIO DE LA SESIÓN DE DEPURACIÓN EN GDB (N = 16)
=====

Punto de interrupción 1 at 0x1988: file asm/vector/
stats_vector.asm, line 92.
Breakpoint 1, compute_stats () at asm/vector/
stats_vector.asm:92
92      push rbx

-----

[EVIDENCIA 1]: Inspección del Registro YMM0 (8 floats)
-----

compute_stats () at asm/vector/stats_vector.asm:93
93      push r12
$1 = {232.468185, 232.468185, 232.468185, 232.468185, 0,
0, 0, 0}

Breakpoint 2, normalize_array () at asm/vector/
stats_vector.asm:237
237      test edx, edx

-----

[EVIDENCIA 2]: Inspección de Memoria del arreglo (vía
$rdi)
-----

238      jle .norm_done
0x55555555a4c0: -56.705162    79.5503387   -14.3966455
               63.4483681
0x55555555a4d0:  70.4831696   -45.020771
               0.653351903   14.4538593
=====
```

V-D. Conclusiones y limitaciones observadas

Los resultados obtenidos muestran que la implementación vectorial mediante AVX2 mantiene la correctitud de los cálculos realizados por la versión escalar, incluyendo los casos borde $N = 0$ y $N = 1$, así como los conjuntos con

valores constantes y valores grandes. Las comparaciones con la referencia en Python presentan errores relativos en órdenes de magnitud bajos, de acuerdo con los resultados documentados en el anexo.

En términos de rendimiento, la versión vectorial reduce el tiempo del kernel en todos los tamaños de entrada evaluados y alcanza el speedup reportado de $13.11\times$ para $N = 10^5$. Para $N = 5 \times 10^7$, los contadores de hardware muestran aproximadamente 1.82×10^9 ciclos y 1.66×10^9 instrucciones para la versión vectorial, frente a 3.46×10^9 ciclos y 3.65×10^9 instrucciones para la versión escalar. Estos resultados evidencian el aprovechamiento del paralelismo SIMD de AVX2.

Como limitación, los resultados de tiempo y los contadores de hardware corresponden al entorno de pruebas descrito, por lo que su comportamiento puede variar con el procesador, la carga del sistema, el compilador y las condiciones de ejecución. Además, la evidencia presentada corresponde a las funciones y tamaños de entrada evaluados en este proyecto.

REFERENCIAS

- [1] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2016. (Convención de llamadas System V AMD64 ABI y preservación de registros).
- [2] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, Vols. 1–2, Intel Corporation, 2023. (Especificación del conjunto de instrucciones AVX2, registros YMM, `vbroadcastss`, `vmovups`, `vminps`, `vmaxps` y la penalización por transición AVX/SSE `vzeroupper`).
- [3] A. Fog, *Instruction Tables: Lists of Instruction Latencies, Throughputs and Micro-Operation Breakdowns for Intel, AMD and VIA CPUs*, Technical University of Denmark, 2023. (Análisis de rendimiento de ciclos de reloj, latencias y optimización de bucles vectoriales).

APÉNDICE

PRUEBAS DE CORRECTITUD Y VERIFICACIÓN DE ESPECIFICACIONES

EVALUANDO CASO DE PRUEBA: input_0.dat

[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):

```

N          = 0
Suma       = 0.000000
Media      = 0.000000
Varianza   = 0.000000
StdDev     = 0.000000
Minimo     = 0.000000
Maximo     = 0.000000
Tiempo promedio del kernel (1 rep.): 0.000104 ms

```

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):

```

N          = 0
Suma       = 0.000000
Media      = 0.000000
Varianza   = 0.000000
StdDev     = 0.000000
Minimo     = 0.000000
Maximo     = 0.000000
Tiempo promedio del kernel (1 rep.): 0.000128 ms

```

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	0.000000	0.000000	0	OK
sum	0.000000	0.000000	0	OK
mean	0.000000	0.000000	0	OK
var	0.000000	0.000000	0	OK
stddev	0.000000	0.000000	0	OK
min	0.000000	0.000000	0	OK
max	0.000000	0.000000	0	OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	0.000000	0.000000	0	OK
sum	0.000000	0.000000	0	OK
mean	0.000000	0.000000	0	OK
var	0.000000	0.000000	0	OK
stddev	0.000000	0.000000	0	OK
min	0.000000	0.000000	0	OK
max	0.000000	0.000000	0	OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL

campo	escalar	vectorial	diff abs	resultado
n	0.000000	0.000000	0.000000e+00	OK
sum	0.000000	0.000000	0.000000e+00	OK
mean	0.000000	0.000000	0.000000e+00	OK
var	0.000000	0.000000	0.000000e+00	OK
stddev	0.000000	0.000000	0.000000e+00	OK
min	0.000000	0.000000	0.000000e+00	OK
max	0.000000	0.000000	0.000000e+00	OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA

EVALUANDO CASO DE PRUEBA: input_1.dat

[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):

```

N          = 1
Suma       = 51.131474
Media      = 51.131474
Varianza   = 0.000000
StdDev     = 0.000000
Minimo     = 51.131474
Maximo     = 51.131474

```

```

Tiempo promedio del kernel (1 rep.): 0.000160 ms

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):
-----
N          = 1
Suma       = 51.131474
Media      = 51.131474
Varianza   = 0.000000
StdDev     = 0.000000
Minimo     = 51.131474
Maximo     = 51.131474
Tiempo promedio del kernel (1 rep.): 0.000175 ms

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON
-----
campo      referencia    obtenido    error rel.  resultado
n          1.000000      1.000000      0          OK
sum        51.131474      51.131473     8.06935e-10 OK
mean       51.131474      51.131473     8.06935e-10 OK
var        0.000000      0.000000      0          OK
stddev     0.000000      0.000000      0          OK
min        51.131474      51.131473     8.06935e-10 OK
max        51.131474      51.131473     8.06935e-10 OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON
-----
campo      referencia    obtenido    error rel.  resultado
n          1.000000      1.000000      0          OK
sum        51.131474      51.131473     8.06935e-10 OK
mean       51.131474      51.131473     8.06935e-10 OK
var        0.000000      0.000000      0          OK
stddev     0.000000      0.000000      0          OK
min        51.131474      51.131473     8.06935e-10 OK
max        51.131474      51.131473     8.06935e-10 OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL
-----
campo      escalar        vectorial    diff abs    resultado
n          0.000000      0.000000      0.000000e+00 OK
sum        0.000000      0.000000      0.000000e+00 OK
mean       0.000000      0.000000      0.000000e+00 OK
var        0.000000      0.000000      0.000000e+00 OK
stddev     0.000000      0.000000      0.000000e+00 OK
min        0.000000      0.000000      0.000000e+00 OK
max        0.000000      0.000000      0.000000e+00 OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA
=====

-----
EVALUANDO CASO DE PRUEBA: input_7.dat
-----

[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):
-----
N          = 7
Suma       = 167.403198
Media      = 23.914743
Varianza   = 1828.749878
StdDev     = 42.763885
Minimo     = -50.202198
Maximo     = 95.004135
Tiempo promedio del kernel (1 rep.): 0.000312 ms

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):
-----
N          = 7
Suma       = 167.403198
Media      = 23.914743
Varianza   = 1828.749878
StdDev     = 42.763885
Minimo     = -50.202198
Maximo     = 95.004135
Tiempo promedio del kernel (1 rep.): 0.000312 ms

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON
-----
campo      referencia    obtenido    error rel.  resultado
n          7.000000      7.000000      0          OK
sum        167.403196      167.403198     9.94701e-09 OK
mean       23.914742      23.914743     4.45939e-08 OK

```

var	1828.749634	1828.749880	1.34355e-07	OK
stddev	42.763882	42.763886	7.34814e-08	OK
min	-50.202198	-50.202198	5.68988e-10	OK
max	95.004135	95.004135	3.35101e-10	OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	7.000000	7.000000	0	OK
sum	167.403196	167.403198	9.94701e-09	OK
mean	23.914742	23.914743	4.45939e-08	OK
var	1828.749634	1828.749880	1.34355e-07	OK
stddev	42.763882	42.763886	7.34814e-08	OK
min	-50.202198	-50.202198	5.68988e-10	OK
max	95.004135	95.004135	3.35101e-10	OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL

campo	escalar	vectorial	diff abs	resultado
n	0.000000	0.000000	0.000000e+00	OK
sum	0.000000	0.000000	0.000000e+00	OK
mean	0.000000	0.000000	0.000000e+00	OK
var	0.000000	0.000000	0.000000e+00	OK
stddev	0.000000	0.000000	0.000000e+00	OK
min	0.000000	0.000000	0.000000e+00	OK
max	0.000000	0.000000	0.000000e+00	OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA

EVALUANDO CASO DE PRUEBA: input_8.dat

[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):

```

N           = 8
Suma        = -73.218163
Media       = -9.152270
Varianza    = 3538.286133
StdDev      = 59.483494
Minimo      = -99.649498
Maximo      = 98.704430
Tiempo promedio del kernel (1 rep.): 0.000283 ms

```

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):

```

N           = 8
Suma        = -73.218178
Media       = -9.152272
Varianza    = 3538.286377
StdDev      = 59.483498
Minimo      = -99.649498
Maximo      = 98.704430
Tiempo promedio del kernel (1 rep.): 0.000292 ms

```

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	8.000000	8.000000	0	OK
sum	-73.218174	-73.218163	1.56801e-07	OK
mean	-9.152272	-9.152270	1.55982e-07	OK
var	3538.286167	3538.286130	1.05463e-08	OK
stddev	59.483495	59.483494	1.87465e-08	OK
min	-99.649498	-99.649498	1.421e-10	OK
max	98.704430	98.704430	2.68122e-10	OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	8.000000	8.000000	0	OK
sum	-73.218174	-73.218178	5.21631e-08	OK
mean	-9.152272	-9.152272	5.16168e-08	OK
var	3538.286167	3538.286380	6.01094e-08	OK
stddev	59.483495	59.483498	4.51367e-08	OK
min	-99.649498	-99.649498	1.421e-10	OK
max	98.704430	98.704430	2.68122e-10	OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL

campo	escalar	vectorial	diff abs	resultado
n	0.000000	0.000000	0.000000e+00	OK
sum	0.000000	0.000000	0.000000e+00	OK
mean	0.000000	0.000000	0.000000e+00	OK
var	0.000000	0.000000	0.000000e+00	OK
stddev	0.000000	0.000000	0.000000e+00	OK
min	0.000000	0.000000	0.000000e+00	OK
max	0.000000	0.000000	0.000000e+00	OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA

EVALUANDO CASO DE PRUEBA: input_15.dat

[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):

```

N           = 15
Suma        = -170.876968
Media       = -11.391798
Varianza    = 3466.972412
StdDev      = 58.881001
Minimo      = -86.072464
Maximo      = 92.991547
Tiempo promedio del kernel (1 rep.): 0.000172 ms

```

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):

```

N           = 15
Suma        = -170.876984
Media       = -11.391799
Varianza    = 3466.971924
StdDev      = 58.880997
Minimo      = -86.072464
Maximo      = 92.991547
Tiempo promedio del kernel (1 rep.): 0.000350 ms

```

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	15.000000	15.000000	0	OK
sum	-170.876976	-170.876968	4.68945e-08	OK
mean	-11.391798	-11.391798	3.51901e-08	OK
var	3466.972133	3466.972410	7.98166e-08	OK
stddev	58.881000	58.881000	1.24897e-08	OK
min	-86.072464	-86.072464	1.24804e-10	OK
max	92.991547	92.991547	3.31851e-10	OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	15.000000	15.000000	0	OK
sum	-170.876976	-170.876984	4.67402e-08	OK
mean	-11.391798	-11.391799	5.25923e-08	OK
var	3466.972133	3466.971920	6.15171e-08	OK
stddev	58.881000	58.880997	5.20472e-08	OK
min	-86.072464	-86.072464	1.24804e-10	OK
max	92.991547	92.991547	3.31851e-10	OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL

campo	escalar	vectorial	diff abs	resultado
n	0.000000	0.000000	0.000000e+00	OK
sum	0.000000	0.000000	0.000000e+00	OK
mean	0.000000	0.000000	0.000000e+00	OK
var	0.000000	0.000000	0.000000e+00	OK
stddev	0.000000	0.000000	0.000000e+00	OK
min	0.000000	0.000000	0.000000e+00	OK
max	0.000000	0.000000	0.000000e+00	OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA

EVALUANDO CASO DE PRUEBA: input_16.dat

```
[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):
-----
N          = 16
Suma       = 232.468201
Media      = 14.529263
Varianza   = 2750.797852
StdDev     = 52.448048
Minimo     = -92.220886
Maximo     = 83.054672
Tiempo promedio del kernel (1 rep.): 0.000430 ms

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):
-----
N          = 16
Suma       = 232.468185
Media      = 14.529262
Varianza   = 2750.798096
StdDev     = 52.448051
Minimo     = -92.220886
Maximo     = 83.054672
Tiempo promedio del kernel (1 rep.): 0.000328 ms

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON
-----
campo      referencia      obtenido      error rel.  resultado
n          16.000000      16.000000      0          OK
sum        232.468208      232.468201      3.19708e-08 OK
mean       14.529263      14.529262      3.62725e-08 OK
var        2750.798153      2750.797850      1.10073e-07 OK
stddev     52.448052      52.448048      8.28374e-08 OK
min        -92.220886      -92.220886      3.30389e-10 OK
max        83.054672      83.054672      4.9619e-10  OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON
-----
campo      referencia      obtenido      error rel.  resultado
n          16.000000      16.000000      0          OK
sum        232.468208      232.468185      1.00797e-07 OK
mean       14.529263      14.529262      9.82164e-08 OK
var        2750.798153      2750.798100      1.91906e-08 OK
stddev     52.448052      52.448051      8.47807e-09 OK
min        -92.220886      -92.220886      3.30389e-10 OK
max        83.054672      83.054672      4.9619e-10  OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL
-----
campo      escalar      vectorial      diff abs      resultado
n          0.000000      0.000000      0.000000e+00  OK
sum        0.000000      0.000000      0.000000e+00  OK
mean       0.000000      0.000000      0.000000e+00  OK
var        0.000000      0.000000      0.000000e+00  OK
stddev     0.000000      0.000000      0.000000e+00  OK
min        0.000000      0.000000      0.000000e+00  OK
max        0.000000      0.000000      0.000000e+00  OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA
=====

-----
EVALUANDO CASO DE PRUEBA: input_1000.dat
-----

[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):
-----
N          = 1000
Suma       = 1975.803955
Media      = 1.975804
Varianza   = 3369.474121
StdDev     = 58.047173
Minimo     = -99.758850
Maximo     = 99.453842
Tiempo promedio del kernel (1 rep.): 0.005018 ms

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):
-----
N          = 1000
Suma       = 1975.805054
Media      = 1.975805
Varianza   = 3369.476562
StdDev     = 58.047192
```

```

Minimo    = -99.758850
Maximo    = 99.453842
Tiempo promedio del kernel (1 rep.): 0.001392 ms

```

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	1000.000000	1000.000000	0	OK
sum	1975.805100	1975.803960	5.77079e-07	OK
mean	1.975805	1.975804	5.72018e-07	OK
var	3369.476359	3369.474120	6.64393e-07	OK
stddev	58.047191	58.047173	3.15095e-07	OK
min	-99.758850	-99.758850	2.34942e-11	OK
max	99.453842	99.453842	3.71168e-10	OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	1000.000000	1000.000000	0	OK
sum	1975.805100	1975.805050	2.54049e-08	OK
mean	1.975805	1.975805	3.04661e-08	OK
var	3369.476359	3369.476560	5.97553e-08	OK
stddev	58.047191	58.047192	1.39474e-08	OK
min	-99.758850	-99.758850	2.34942e-11	OK
max	99.453842	99.453842	3.71168e-10	OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL

campo	escalar	vectorial	diff abs	resultado
n	0.000000	0.000000	0.000000e+00	OK
sum	0.000000	0.000000	0.000000e+00	OK
mean	0.000000	0.000000	0.000000e+00	OK
var	0.000000	0.000000	0.000000e+00	OK
stddev	0.000000	0.000000	0.000000e+00	OK
min	0.000000	0.000000	0.000000e+00	OK
max	0.000000	0.000000	0.000000e+00	OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA

EVALUANDO CASO DE PRUEBA: input_const.dat

[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):

```

N          = 100
Suma      = 500.000000
Media     = 5.000000
Varianza  = 0.000000
StdDev    = 0.000000
Minimo    = 5.000000
Maximo    = 5.000000
Tiempo promedio del kernel (1 rep.): 0.000291 ms

```

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):

```

N          = 100
Suma      = 500.000000
Media     = 5.000000
Varianza  = 0.000000
StdDev    = 0.000000
Minimo    = 5.000000
Maximo    = 5.000000
Tiempo promedio del kernel (1 rep.): 0.000321 ms

```

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	100.000000	100.000000	0	OK
sum	500.000000	500.000000	0	OK
mean	5.000000	5.000000	0	OK
var	0.000000	0.000000	0	OK
stddev	0.000000	0.000000	0	OK
min	5.000000	5.000000	0	OK
max	5.000000	5.000000	0	OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	100.000000	100.000000	0	OK
sum	500.000000	500.000000	0	OK
mean	5.000000	5.000000	0	OK
var	0.000000	0.000000	0	OK
stddev	0.000000	0.000000	0	OK
min	5.000000	5.000000	0	OK
max	5.000000	5.000000	0	OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL

campo	escalar	vectorial	diff abs	resultado
n	0.000000	0.000000	0.000000e+00	OK
sum	0.000000	0.000000	0.000000e+00	OK
mean	0.000000	0.000000	0.000000e+00	OK
var	0.000000	0.000000	0.000000e+00	OK
stddev	0.000000	0.000000	0.000000e+00	OK
min	0.000000	0.000000	0.000000e+00	OK
max	0.000000	0.000000	0.000000e+00	OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA

EVALUANDO CASO DE PRUEBA: input_edge.dat

[1] RESULTADO SALIDA ESCALAR (bin/norm_scalar):

```

N           = 100
Suma        = 0.000000
Media       = 0.000000
Varianza    = 299999952896.000000
StdDev      = 547722.500000
Minimo      = -1000000.000000
Maximo      = 1000000.000000
Tiempo promedio del kernel (1 rep.): 0.000844 ms

```

[2] RESULTADO SALIDA VECTORIAL (bin/norm_vector):

```

N           = 100
Suma        = 0.000000
Media       = 0.000000
Varianza    = 299999985664.000000
StdDev      = 547722.562500
Minimo      = -1000000.000000
Maximo      = 1000000.000000
Tiempo promedio del kernel (1 rep.): 0.000380 ms

```

[3] COMPARACIÓN: ESCALAR vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	100.000000	100.000000	0	OK
sum	0.000000	0.000000	0	OK
mean	0.000000	0.000000	0	OK
var	300000000000.280029299999953000	0.000000	1.56668e-07	OK
stddev	547722.557505	547722.500000	1.0499e-07	OK
min	-1000000.000000	-1000000.000000	0	OK
max	1000000.000000	1000000.000000	0	OK

RESULTADO GENERAL: PASA

[4] COMPARACIÓN: VECTORIAL vs REFERENCIA PYTHON

campo	referencia	obtenido	error rel.	resultado
n	100.000000	100.000000	0	OK
sum	0.000000	0.000000	0	OK
mean	0.000000	0.000000	0	OK
var	300000000000.280029299999986000	0.000000	4.66676e-08	OK
stddev	547722.557505	547722.562000	8.20594e-09	OK
min	-1000000.000000	-1000000.000000	0	OK
max	1000000.000000	1000000.000000	0	OK

RESULTADO GENERAL: PASA

[5] COMPARACIÓN DIRECTA: ESCALAR vs VECTORIAL

campo	escalar	vectorial	diff abs	resultado
n	0.000000	0.000000	0.000000e+00	OK
sum	0.000000	0.000000	0.000000e+00	OK
mean	0.000000	0.000000	0.000000e+00	OK
var	0.000000	0.000000	0.000000e+00	OK

stddev	0.000000	0.000000	0.000000e+00	OK
min	0.000000	0.000000	0.000000e+00	OK
max	0.000000	0.000000	0.000000e+00	OK

RESULTADO COMPARACIÓN ESCALAR vs VECTORIAL: PASA